Анализ данных с использованием языка Python

Пояснения к коду. Семинар 1

Введение в курс: Jupyter Lab, Markdown, функции Python

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-01-tasks.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху; в первом разделе он совпадает с комментарием «Это ячейка N». Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда. Разобран только готовый код ноутбука: ячейки первого раздела, заготовки функций, тесты и проверки. Тела функций — задача теста «Семинар 1», решений здесь нет. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены.

1 Работа с ячейками в Jupyter Lab

Ячейка 1. Подключение NumPy

```
1 # Это ячейка 1.
2
3 import numpy as np
```

– `# Это ячейка 1.` — комментарий: всё, что стоит в строке после решётки, Python не выполняет. Здесь комментарий нумерует ячейку.

– `import numpy as np` — подключить библиотеку NumPy под коротким именем `np`: дальше её функции вызываются как `np.имя`.

Ячейка 2. Генератор и пять исков

```
1 # Это ячейка 2.
2
3 rng = np.random.default_rng(42)  # генератор случайных чисел с зерном 42
4 amounts = rng.integers(10_000, 500_000, size=5)  # суммы пяти исков за ↩
  ↪ день, руб.
```

– `rng = np.random.default_rng(42)` — генератор случайных чисел; 42 — зерно. С одним и тем же зерном генератор выдаёт одни и те же числа, поэтому результат одинаков у всех.

– `rng.integers(10_000, 500_000, size=5)` — пять случайных целых чисел от 10 000 включительно до 500 000, сама верхняя граница не входит. Подчёркивание внутри числа — разделитель разрядов для удобства чтения, на значение он не влияет.

– `amounts = ...` — результат записывается в переменную `amounts`: это массив NumPy из пяти чисел, он хранится в памяти ядра.

Ячейка 3. Три функции

```
# Это ячейка 3.

def show_mean():
    """Печатает среднюю сумму иска по массиву amounts из текущего ↩
↪ состояния ноутбука, округлённую до рубля."""
    print(round(amounts.mean()))


def add_claim(claims, generator):
    """Возвращает массив исков с одним новым иском на случайную сумму."""
    return np.append(claims, generator.integers(10_000, 500_000))


def new_day(generator):
    """Возвращает пять исков нового дня."""
    return generator.integers(10_000, 500_000, size=5)
```

– `def show_mean():` — заголовок функции: имя, пустые скобки — параметров нет, двоеточие. Тело функции — строки с отступом под заголовком.

– Строка в тройных кавычках сразу под заголовком — описание функции. Python его не выполняет, оно для читателя.

– `print(round(amounts.mean()))` — среднее массива `amounts`, округлённое до целого, на экран. Функция берёт `amounts` из памяти ядра в момент вызова: что лежит в памяти, то и посчитает.

– `def add_claim(claims, generator):` — функция с двумя параметрами: массив исков и генератор. `generator.integers(10_000, 500_000)` без `size` — одно случайное число.

– `np.append(claims, ...)` — новый массив: старые числа и одно новое в конце. Сам массив `claims` не меняется.

– `return` — что функция отдаёт тому, кто её вызвал. У `show_mean` его нет: она только печатает.

– `def new_day(generator):` — функция возвращает пять новых случайных чисел.

Ячейка 4. Вызов функции

```
# Это ячейка 4.

show_mean()
```

– `show_mean()` — вызов функции: имя и скобки. Без скобок функция не выполнится.

Ячейка 5. Новый иск

```
# Это ячейка 5.

amounts = add_claim(amounts, rng)
```

– `amounts = add_claim(amounts, rng)` — функция возвращает новый массив, и он записывается в ту же переменную: старое значение `amounts` заменяется новым. Каждый запуск ячейки добавляет ещё один иск.

Ячейка 6. Снова среднее

```
# Это ячейка 6.

show_mean()
```

– Тот же вызов, что в ячейке 4. Напечатанное число зависит от того, что сейчас лежит в `amounts`.

Ячейка 7. Генератор без зерна

```
# Это ячейка 7.

rng = np.random.default_rng()  # генератор без зерна
amounts = new_day(rng)
```

– `np.random.default_rng()` без зерна — генератор, который при каждом создании выдаёт новые числа.

– `amounts = new_day(rng)` — пять исков нового дня вместо прежнего массива.

2 Оценка точности предсказаний

Ячейка 8. Заготовка функции

```
def accuracy_solution(real, predicted):
    """
    Принимает список фактических исходов дел и список предсказаний программы,
    возвращает точность программы: долю дел, для которых предсказание
    совпало с фактическим исходом.

    Аргументы:
        real: Список из 0 и 1: 1 означает, что иск удовлетворён, 0 - что в ↩
↪ иске отказано.
        predicted: Список из 0 и 1 той же длины с предсказаниями программы
                   для каждого дела в том же порядке.

    Возвращаемое значение:
        Доля дел с верным предсказанием: число совпадений, делённое
        на общее число дел. Например, 42 совпадения из 100 дают 0.42.
    """

    pass
```

– `def accuracy_solution(real, predicted):` — заготовка: имя и два параметра, два списка.

– Описание в тройных кавычках говорит, что функция принимает («Аргументы») и что возвращает («Возвращаемое значение»).

– `pass` — пустое тело: функция ничего не делает и возвращает `None`. Вместо `pass` пишется тело функции.

Ячейка 9. Тесты

```
def accuracy_tests():
    rng = np.random.default_rng(1)
    example_1_real = rng.integers(0, 2, size=100).tolist()
    example_1_predicted = rng.integers(0, 2, size=100).tolist()
    assert accuracy_solution(example_1_real, example_1_predicted) == 0.54

    example_2_real = rng.integers(0, 2, size=1000).tolist()
    example_2_predicted = rng.integers(0, 2, size=1000).tolist()
    assert accuracy_solution(example_2_real, example_2_predicted) == 0.517

    example_3_real = rng.integers(0, 2, size=500).tolist()
    example_3_predicted = rng.integers(0, 2, size=500).tolist()
    assert accuracy_solution(example_3_real, example_3_predicted) == 0.474

    print('Все тесты прошли успешно!')
```

– `def accuracy_tests():` — тесты тоже собраны в функцию; ячейка только определяет её, а запускает ячейка 10.

– `np.random.default_rng(1)` — генератор с зерном 1: тестовые примеры одинаковы у всех.

– `rng.integers(0, 2, size=100)` — сто чисел, каждое 0 или 1: верхняя граница 2 не входит. `.tolist()` — массив NumPy в обычный список Python: функция получает списки, как сказано в описании.

– `assert ... == 0.54` — проверка: вызвать функцию и сравнить результат с известным ответом. Если сравнение ложно, Python останавливается с ошибкой `AssertionError` на этой строке.

– `print('Все тесты прошли успешно!')` — последняя строка; до неё доходит, только если ни одна проверка не остановила функцию.

Ячейка 10. Запуск тестов

```
accuracy_tests()
```

– `accuracy_tests()` — запуск тестов. Пока вместо тела стоит `pass`, тест падает на первом `assert`.

Ячейка 11. Проверка

```
# Проверка: вызов функции на двадцати делах для ответа в тесте Moodle

real_check      = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0]
predicted_check = [1, 0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0]

print(accuracy_solution(real_check, predicted_check))
```

– Строки 3–4 — два списка в квадратных скобках, элементы через запятую. Пробелы после `real_check` выравнивают списки один под другим и на результат не влияют.

– `print(accuracy_solution(real_check, predicted_check))` — вызвать функцию на этих списках и напечатать результат. Пока функция не дописана, печатается `None`.

3 Смены стадий дела

Ячейка 12. Заготовка функции

```
def remove_adjacent_solution(numbers):
    """
    Принимает список кодов стадий дела по дням и возвращает новый список,
    в котором удалены повторы соседних элементов: последовательность смен стадий.

    Например, список [1, 1, 2, 2, 2, 3, 4, 3, 3, 5] будет преобразован
    в список [1, 2, 3, 4, 3, 5].

    Аргументы:
        numbers: Список кодов стадий по дням, может быть пустым.

    Возвращаемое значение:
        Новый список без повторов соседних элементов.
        Для пустого списка - пустой список.
    """

    pass
```

– Заготовка с одним параметром — списком **numbers**. Пример в описании показывает, что должно получиться.

Ячейка 13. Тесты

```
def remove_adjacent_tests():
    assert remove_adjacent_solution([1, 1, 2, 2, 2, 3, 4, 3, 3, 5]) == [1, 2, 3, 4, 3, 5]
    assert remove_adjacent_solution([5, 5, 5]) == [5]
    assert remove_adjacent_solution([1, 2, 3]) == [1, 2, 3]
    assert remove_adjacent_solution([]) == []

    rng = np.random.default_rng(2)
    example_1 = rng.integers(1, 6, size=100).tolist()
    result_1 = remove_adjacent_solution(example_1)
    assert len(result_1) == 80
    assert result_1[:12] == [5, 2, 1, 2, 3, 5, 3, 1, 2, 4, 5, 4]
    assert result_1[-6:] == [3, 1, 2, 1, 3, 1]
    for i in range(1, len(result_1)):
        assert result_1[i] != result_1[i - 1]

    example_2 = rng.integers(1, 6, size=1000).tolist()
    result_2 = remove_adjacent_solution(example_2)
    assert len(result_2) == 781
    assert result_2[:12] == [3, 4, 3, 2, 5, 2, 5, 3, 2, 4, 5, 4]
    assert result_2[-6:] == [4, 3, 2, 5, 4, 5]
    for i in range(1, len(result_2)):
        assert result_2[i] != result_2[i - 1]

    print('Все тесты прошли успешно!')
```

– Строки 2–5 — проверки на маленьких списках: списки сравниваются поэлементно и по порядку. `[]` — пустой список.

– `rng.integers(1, 6, size=100).tolist()` — сто случайных кодов от 1 до 5.

– `len(result_1) == 80` — длина результата; `len` — число элементов списка.

– `result_1[:12]` — первые двенадцать элементов, `result_1[-6:]` — последние шесть: срезы списка.

– `for i in range(1, len(result_1)):` — цикл по номерам элементов, начиная со второго: `range(1, n)` — числа от 1 до n без самого n. Нумерация в Python начинается с нуля.

– `assert result_1[i] != result_1[i - 1]` — каждый элемент не равен предыдущему; `!=` — «не равно».

Ячейка 14. Запуск тестов

```
1 remove_adjacent_tests()
```

– Запуск тестов, как в ячейке 10.

Ячейка 15. Проверка

```
1 # Проверка: коды стадий за 40 дней для ответа в тесте Moodle
2
3 stages_check = [5, 4, 4, 5, 3, 4, 5, 2, 1, 2, 2, 5, 5, 1, 3, 5, 1, 4, 1, 3,
4                 5, 2, 2, 2, 4, 2, 5, 3, 3, 3, 3, 3, 3, 5, 5, 4, 4, 4, 2, 5]
5
6 print(stages_check)
7 stages = remove_adjacent_solution(stages_check)
8 print(stages)
9 print(len(stages))
```

– Строки 3–4 — список: коды стадий за 40 дней. Он записан на двух строках: пока квадратная скобка не закрыта, Python ждёт продолжения.

– Строки 6–9 — напечатать исходный список, записать результат функции в `stages` и напечатать его и его длину. Пока функция не дописана, результат — `None`, а `len(None)` падает с ошибкой `TypeError`.

4 Подсчёт упоминаний слова

Ячейка 16. Заготовка: разбить текст на слова

```python
def string_to_words_solution(s):
    """
    Разделяет подготовленный текст на слова.

    Аргументы:
        s: Непустая строка из слов, разделённых одним пробелом, без знаков препинания,
           без пробелов в начале и в конце.

    Возвращаемое значение:
        Список слов, входящих в строку, в том же порядке,
        в котором они встречались в исходной строке.
    """

    pass
```

– Заготовка с одним параметром — строкой `s`. Описание оговаривает, какой текст приходит: слова через один пробел, без знаков препинания.

Ячейка 17. Слова для тестовых текстов

```python
VOCAB = ['суд', 'судья', 'иск', 'исковое', 'заявление', 'ответчик', 'ответчика', 'истец', 'решение', 'отказать', 'удовлетворить', 'договор', 'неустойка', 'заседание']  # слова для тестовых текстов
```

– `VOCAB = [...]` — список строк, из которых тесты собирают тексты. Имя большими буквами — обычай для значения, которое не меняется.

– Ячейка отдельная: если её пропустить, тесты упадут с ошибкой `NameError` про `VOCAB`.

Ячейка 18. Тесты

```python
def string_to_words_tests():
    assert string_to_words_solution('суд установил что ответчик не явился') == ['суд', 'установил', 'что', 'ответчик', 'не', 'явился']
    assert string_to_words_solution('решение') == ['решение']

    rng = np.random.default_rng(3)
    example_1 = ' '.join(rng.choice(VOCAB, size=20).tolist())
    words_1 = string_to_words_solution(example_1)
    assert len(words_1) == 20
    assert words_1[:5] == ['договор', 'судья', 'иск', 'исковое', 'иск']
    assert words_1[-3:] == ['удовлетворить', 'суд', 'судья']

    example_2 = ' '.join(rng.choice(VOCAB, size=100).tolist())
    words_2 = string_to_words_solution(example_2)
    assert len(words_2) == 100
    assert words_2[:5] == ['ответчика', 'ответчик', 'неустойка', 'истец', 'ответчик']
    assert words_2[-3:] == ['решение', 'заседание', 'иск']

    example_3 = ' '.join(rng.choice(VOCAB, size=1000).tolist())
    words_3 = string_to_words_solution(example_3)
```

```
20      assert len(words_3) == 1000
21      assert words_3[:5] == ['договор', 'иск', 'суд', 'удовлетворить', 'иск']
22      assert words_3[-3:] == ['договор', 'исковое', 'решение']
23
24      print('Все тесты прошли успешно!')
```

– Строки 2–3 — проверки на двух готовых строках. Строка записывается в одинарных кавычках.

– `rng.choice(VOCAB, size=20)` — двадцать случайных слов из списка, слова могут повторяться.

– `' '.join(...)` — склеить список слов в одну строку через пробел: так получается тестовый текст.

– `words_1[:5]` и `words_1[-3:]` — первые пять и последние три слова результата.

Ячейка 19. Запуск тестов

```
1  string_to_words_tests()
```

– Запуск тестов.

Ячейка 20. Заготовка: подсчитать слово

```
1  def count_word_solution(s, word):
2      """
3      Подсчитывает, сколько раз в подготовленном тексте s встречается слово ↩
   ↪ word.
4      Слова считаются целиком: «ответчик» и «ответчика» - разные слова.
5
6      Аргументы:
7          s: Строка из слов, разделённых одним пробелом.
8          word: Слово, упоминания которого нужно подсчитать.
9
10     Возвращаемое значение:
11         Число упоминаний слова word в тексте s; 0, если слово не встречается.
12     """
13
14     words = string_to_words_solution(s)
15
16     pass
```

– Заготовка с двумя параметрами: текст `s` и слово `word`.

– Строка 14 уже написана: `words = string_to_words_solution(s)` — функция вызывает функцию из прошлой задачи и сначала разбивает текст на слова. Поэтому без неё эта задача не решается.

Ячейка 21. Тесты

```
1  def count_word_tests():
2      assert count_word_solution('ответчик извещён ответчика уведомили', ↩
   ↪ 'ответчик') == 1
3      assert count_word_solution('иск подан иск принят', 'иск') == 2
4      assert count_word_solution('суд установил', 'судья') == 0
5
6      rng = np.random.default_rng(3)
7      example_1 = ' '.join(rng.choice(VOCAB, size=20).tolist())
8      assert count_word_solution(example_1, 'суд') == 2
9
10     example_2 = ' '.join(rng.choice(VOCAB, size=100).tolist())
11     assert count_word_solution(example_2, 'иск') == 7
12
13     example_3 = ' '.join(rng.choice(VOCAB, size=1000).tolist())
14     assert count_word_solution(example_3, 'ответчик') == 78
15
16     print('Все тесты прошли успешно!')
```

– Генератор с тем же зерном 3, что в ячейке 18: тексты получаются те же.

– `== 0` — проверка случая, когда слова в тексте нет.

Ячейка 22. Запуск тестов

```
1  count_word_tests()
```

– Запуск тестов.

Ячейка 23. Проверка

```
1  # Проверка: фраза из решения суда для ответа в тесте Moodle
2
3  text_check = 'ответчик в судебное заседание не явился ответчик извещён ↩
   ↪ надлежащим образом суд рассмотрел дело в отсутствие ответчика'
4
5  print(count_word_solution(text_check, 'ответчик'))
```

– `text_check` — одна длинная строка в одинарных кавычках.

– Строка 5 — сколько раз в этой строке встречается слово «ответчик» целиком.

5 Разбиение строки по разделителю

Ячейка 24. Заготовка функции

```python
def split_line_solution(line, sep):
    """
    Разбивает строку по разделителю и возвращает список значений между
    разделителями. Если значение обособлено двойными кавычками, разделитель
    внутри него не считается разделителем, а сами кавычки в результат не входят.

    Например, для строки 'Иванов Иван,А40-1/2026,"иск, встречный иск",2026'
    и разделителя ',' результат ['Иванов Иван', 'А40-1/2026', 'иск, встречный иск', '2026'].
    Для пустой строки результат [''] - список из одной пустой строки.

    Аргументы:
        line: Строка, которую нужно разбить; двойные кавычки в ней встречаются
              только парами, обособляющими целое значение.
        sep: Разделитель, строго один символ; гарантируется, что это не двойная кавычка.

    Возвращаемое значение:
        Список значений, полученных разбиением строки по разделителю.
    """

    pass
```

– Заготовка с двумя параметрами: строка `line` и разделитель `sep`. Пример в описании записан в одинарных кавычках, поэтому двойные кавычки внутри него — часть строки.

Ячейка 25. Тесты

```python
def split_line_tests():
    example_1_line = 'Иванов Иван,А40-1/2026,"иск, встречный иск",2026'
    example_1_res = ['Иванов Иван', 'А40-1/2026', 'иск, встречный иск', '2026']

    assert split_line_solution(example_1_line, ',') == example_1_res

    example_2_line = 'Петров Пётр;;2025'
    example_2_res = ['Петров Пётр', '', '2025']

    assert split_line_solution(example_2_line, ';') == example_2_res

    example_3_line = 'Сидорова Анна,"[запись 1, запись 2, запись 3]", ,2'
    example_3_res = ['Сидорова Анна', '[запись 1, запись 2, запись 3]', ' ', '2']

    assert split_line_solution(example_3_line, ',') == example_3_res

    example_4_line = 'А40-1/2026,,'
    example_4_res = ['А40-1/2026', '', '']

    assert split_line_solution(example_4_line, ',') == example_4_res

    example_5_line = 'А40-7/2026,"неустойка, проценты"'
    example_5_res = ['А40-7/2026', 'неустойка, проценты']

```

```
    assert split_line_solution(example_5_line, ',') == example_5_res

    example_6_line = 'A40-1/2026,"",2026'
    example_6_res = ['A40-1/2026', '', '2026']

    assert split_line_solution(example_6_line, ',') == example_6_res

    assert split_line_solution('', ',') == ['']

    print('Все тесты прошли успешно!')
```

– Каждая проверка — пара: строка `example_N_line` и ожидаемый список `example_N_res`.

– `''` — пустая строка, `['']` — список из одной пустой строки.

Ячейка 26. Запуск тестов

```
split_line_tests()
```

– Запуск тестов.

Ячейка 27. Проверка

```
# Проверка: строка из выгрузки картотеки для ответа в тесте Moodle

line_check = 'Петров Пётр,A40-7/2026,"неустойка, проценты, ↩
↪ штраф",250000,2026'

values = split_line_solution(line_check, ',')
print(values)
print(len(values))
```

– `values = split_line_solution(line_check, ',')` — результат в переменную; дальше печатаются список и его длина.